TRAVAUX PRATIQUES

Dominante INFONUM CentraleSupélec - 2026 -2027

Prendre en main et utiliser de façon critique et productive un environnement de développement augmenté par l'IA

Ce tp a été rédigé en faisant appel à des outils d'IA Générative (Claude - Sonnet 5 High).

1 Objectifs pédagogiques

L'objectif principal de la séance est de mettre en place un environnement de développement augmenté par une IA générative et de l'utiliser, de manière critique, dans une tâche de développement de logiciels.

Objectifs d'apprentissage :
A l'issue de la séance, vous devriez être capable de :

- Configurer et prendre en main un environnement de développement augmenté par une IA générative intégrée à l'IDE ;
- Décomposer une tâche de développement pour l'exploiter efficacement avec un assistant IA (formulation des demandes, découpage en sous-tâches, itération) ;
- Évaluer de façon critique la pertinence, la fiabilité et les limites des suggestions produites par l'IA (relecture, tests, détection d'erreurs et d'« hallucinations ») ;
- Documenter et justifier ses choix d'usage de l'IA de façon traçable et transparente ;
- Identifier et discuter les enjeux de souveraineté des données, de sécurité, de propriété du code et de dépendance à l'outil liés à l'usage d'une IA générative en contexte professionnel.

Dans ce travail, nous nous appuierons sur Aristote [1], qui donne accès à un service de modèles LLM souverains, et sur l'infrastructure d'IA mutualisée ILaaS [2].

Vous avez normalement chacun reçu un lien secret vous permettant d'accéder à vos clés d'accès. Vous signaler si ce n'est pas le cas

Important

Ce TP ne vise pas à évaluer la performance brute des modèles fournis par ILaaS. Ces modèles ne sont pas aussi puissants que les outils commerciaux. L'objet de ce travail est d'une part de s'initier et de prendre en main les outils et aussi de travailler à avoir une posture méthodologique et critique face à ces outils, quels qu'ils soient.
En entreprise, l'assistance par IA générative est déjà la norme. La compétence qui fait la différence n'est plus *savoir utiliser un assistant IA* mais savoir quand lui faire confiance, quand le vérifier, quand s'en passer, et comment rester **responsable** du résultat produit.

2 Mission

Par binôme, vous allez traiter une tâche réelle de développement logiciel sur un dépôt de votre choix, en utilisant Aristote comme assistant intégré à votre IDE. Une liste de dépôt est proposée pour chaque mention. L'objectif n'est pas seulement de livrer un code fonctionnel, mais aussi de documenter et d'analyser la manière dont vous avez travaillé avec l'IA.

1. `https://aristote.pages.centralesupelec.fr/aristote-user-docs/`
2. `https://www.ilaas.fr/`

Règles du jeu

- L'usage de l'IA est encouragé et attendu — l'objectif est de l'utiliser intensivement, pas de s'en passer ;
- Toute suggestion acceptée doit être relue et comprise avant intégration : vous restez responsables de tout le code rendu, IA ou non ;
- Toute modification issue de l'IA doit être testée (tests existants + tests ajoutés si pertinent) avant d'être considérée comme acquise ;
- Vous devrez renseigner un journal de prompts (voir en annexe) en continu.
- Vous documenterez explicitement au moins deux interactions où l'IA s'est trompée, a halluciné, ou a proposé une solution inadaptée : ce sont ces cas qui nourriront le mieux votre analyse.

Livrables attendus

Chaque groupe devra rendre

- Le code produit (commité avec des messages de commit clairs) : partage de dépôt github.
- Son journal de prompts (EDUNAO).
- La fiche d'analyse critique remplie en séance (EDUNAO).
- Une note de prise de recul (1 à 2 pages, à rendre sous une semaine) répondant à : qu'est-ce que l'IA vous a fait gagner ou perdre ? quelles limites avez-vous rencontrées et comment les avez-vous gérées ? quels enjeux (souveraineté, sécurité, dépendance, propriété du code) retenez-vous de cette expérience ?

3 Configuration et mise en place des outils

Vous travaillerez avec vos machines personnelles avec comme configuration standard minimale :

- **IDE** : Visual Studio Code (dernière version stable)
- Extensions VS Code : Python, Opencode [3].
- Python, versions supérieures à 3.10. Les versions recommandées sont dépendantes des dépôts sur lesquels vous allez travailler. Une version recommandée reste cependant 3.12.
- Il est fortement recommandé d'utiliser des environnements virtuels [4] par dépôt et tâche.

```
python3 -m venv .venv
```

- Git : dernière version stable.
- OpenCode CLI : installée et mise à jour. Voir `https://opencode.ai/` pour l'installation

Rappel

OpenCode est un agent de code open source conçu spécifiquement pour l'assistance au développement. Il s'utilise en ligne de commande ou via son extension officielle dans VS Code. Il se configure pour pointer vers un fournisseur de modèle personnalisé compatible OpenAI (fichier opencode.json), ce qui permet de le brancher directement ici sur l'API Aristote.
Une alternative pourrait être OpenClaw [a], autre projet open source mais de nature très différente : c'est un assistant personnel « toujours actif », exécuté en arrière-plan comme un démon avec sa propre boucle de planification, doté d'une mémoire persistante et capable d'agir de façon autonome, sans invite explicite, sur l'e-mail, l'agenda, les fichiers ou des messageries. Il est donc à utiliser avec plus de prudence et il répond moins aux objectifs pédagogiques de cette activité.

a. `https://openclaw.ai/`

3. `https://opencode.ai/`
4. voir `https://realpython.com/python-virtual-environments-a-primer/`

3.1 Connexion à Aristote (une fois la configuration standard installée)

— Créer, à la racine du dépôt de travail, un fichier opencode.json déclarant un provider personnalisé compatible OpenAI (type @ai-sdk/openai-compatible) avec pour baseURL le point d'accès de l'API Aristote : `https://llm.aristote.education/v1`.

— Déclarer dans ce même fichier le ou les modèles Aristote disponibles :

— Qwen3.6-35B-A3B-NVFP4 [5]

— Qwen3.8-Flash-Next-NVFP4 [6]

— gemma-4-31B-it-NVFP4 [7]

—

Plusieurs modèles vous sont fournis. **Il vous appartient de vous renseigner sur ces modèles et il est attendu d'essayer ces différents modèles avec un choix avisé le cas échéant.**

```
{
{
  "$schema": "https://opencode.ai/config.json",
  "model": "aristote/qwen-3.6-35b-instruct",
  "small_model": "aristote/qwen-3.6-35b-instruct",
  "autoupdate": "notify",
  "compaction": {
    "auto": true,
    "prune": true,
    "reserved": 8192
  },
  "tools": {
    "task": false,
    "webfetch": false
  },
  "provider": {
    "aristote": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "Aristote",
      "options": {
        "baseURL": "https://llm.aristote.education/v1",
        "apiKey": "sk...    compl ter",
        "timeout": 600000,
        "chunkTimeout": 60000
      },
      "models": {
        "qwen-3.6-35b-instruct": {
          "name": "Qwen 3.6 35B Instruct",
          "limit": {
            "context": 262144,
            "output": 65536
          }
        },
        "gemma-4-31b": {
          "name": "Gemma 4 31B",
          "limit": {
            "context": 131072,
            "output": 16384
          }
        }
      }
    }
  }
```

5. https ://huggingface.co/RedHatAI/Qwen3.6-35B-A3B-NVFP4
6. https ://huggingface.co/primitive-ai/Qwen3.8-Flash-Next-NVFP4
7. https ://huggingface.co/RedHatAI/gemma-4-31B-it-NVFP4

```
}
```

Ne copier/coller pas ce fichier de configuration. Vous pouvez le récupérer directement depuis le site d'Aristote ici

- Méthode d'authentification et de récupération de la clé/du jeton Aristote, à enregistrer via la commande /connect d'OpenCode (stockage local dans auth.json)
- Vérifier que le modèle Aristote apparaît bien dans le sélecteur de modèles d'OpenCode.
- Tester la configuration à l'aide de la commande ci-dessous, à exécuter dans le terminal intégré de VS Code à la racine du dépôt contenant `opencode.json`.

```
opencode models
```

3.2 Exercice 1 : Exercice de prise en main OpenCode

Objectifs d'apprentissage : Se familiariser avec les mécaniques d'OpenCode sur un mini-projet jouet, sans enjeu de compréhension d'un vrai dépôt.

Créér un dossier projet sur votre poste et telecharger l'archive TutoOpenCode disponible sur Edunao. Cette archive contient deux fichiers de départ :

- toolbox.py
- test_toolbox.py

Vous utilisez la bibliothèque pytest [8] pour faire vos tests qui doit donc être installée.

- Get Started
- Petit tuto

Voici les différentes étapes à suivre.

1. Prendre connaissance du code, le tester, l'exécuter. Lancer `pytest`.
2. Se placer dans son répertoire de travail, lancer `opencode` puis `\init`. `opencode` qui devrait générer un fichier `AGENTS.md`. Prendre connaissance de ce fichier et vérifier qu'il décrit correctement le projet.
3. **Tâche 1 : Diagnostiquer et corriger un bug connu**. Lors du lancement de `pytest`, un test a échoué. Utiliser `opencode` pour comprendre et corriger la fonction `is_palindrome`. Ici, il conviendra d'essayer plusieurs configurations pour se familiariser avec Opencode. Vous pouvez par exemple lui demander *"Le test test_is_palindrome_with_spaces échoue, explique pourquoi et corrige is_palindrome"* mais vous pouvez aussi le faire de manière incrémentale. Dans tous les cas, il faudra observer le `diff`, l'accepter si il vous convient et relancer `pytest`pour confirmer.
4. **Tâche 2 : Écrire une fonction à partir d'une specification**. Ici on travaillera sur la fonction `word_frequency(text)`. Utiliser `opencode`pour implémenter cette fonction. Ici aussi vous pouvez essayer différentes consignes en augmentant à chaque fois le niveau de la spécification comme par exemple :
   - *"Implémente word_frequency(text)"*
   - *"Implémente word_frequency(text) :dictionnaire mot : occurrences"*
   - *"Implémente word_frequency(text) :dictionnaire mot : occurrences,insensible à la casse, en ignorant la ponctuation."*
   - ...

   Il conviendra de vérifier le résultat à la main sur un exemple avant d'accepter.

   Nous sommes ici sur une bonne illustration du concept de **software 3.0** popularisé par Andrej Karpathy, selon laquelle l'anglais devient le langage de programmation : Écrivez ce que vous voulez en langage naturel, et obtenez un code fonctionnel [9].

8. `https://docs.pytest.org/en/stable/`
9. `https://www.software-3.com/`

5. **Tâche 3 : Écrire des tests** - Utiliser `opencode`pour écrire les tests permettant de tester la fonction `word_frequency`. Il faudra ensuite lancer pytest et juger si les cas limites sont vraiment couverts.
6. **Tâche 4 : tester l'annulation**. Demander à `opencode`une modification volontairement sous-spécifiée (comme par exemple *"rends is_palindrome plus rapide"*, sans autre précision). Observer ce que l'IA décide d'elle-même, puis faire la commande `\undo`si le résultat ne convient pas ;
7. **Tâche 5 : comparer plusieurs modèles** Comparer plusieurs modèles. Pour cela, vous pouvez utiliser la commande `\models`. Elle permet de lister les différents modèles. Changer de modèle Aristote, demander à `opencode`d'écrire la fonction celsius_to_fahrenheit avec le même prompt, comparer les résultats.

Rappel

Ces exercices permettent de pratiquer les concepts dit de **software 3.0**, popularisé par Andrej Karpathy ou de**Harness Engineering**, popularisé et formalisé principalement par OpenAI (ainsi que par l'ingénieur Mitchell Hashimoto. Ces deux concepts décrivent le même virage concernant le developpement sous deux angles complémentaires. Karpathy pose le cadre conceptuel : après le code explicite (1.0) et les poids appris (2.0), on programme désormais en langage naturel, où "la fenêtre de contexte est le programme, le LLM est l'interpréteur". Le harness engineering, lui, est la réponse d'ingénierie pratique à ce constat : puisqu'un interpréteur seul ne suffit pas, il faut construire tout ce qui l'entoure — outils, mémoire, sandbox, garde-fous, boucles de feedback (guides en amont, sensors en aval) — pour que l'agent agisse de façon fiable dans un vrai système. Les deux courants s'accordent sur **un même déplacement du travail d'ingénieur** : on écrit moins de code directement, et on conçoit davantage l'environnement et les systèmes de contrôle qui pilotent l'agent. Software 3.0 dit quoi a changé (une nouvelle unité de calcul programmable en langage naturel) ; le harness engineering dit comment la rendre exploitable et robuste en production.
Pour aller plus loin :
— Ning et al, 2026 - Code as Agent Harness. [a]
— `https://harness-engineering.ai/`

a. `https://github.com/YennNing/Awesome-Code-as-Agent-Harness-Papers`

3.3 Exercice 2 : Exercice de prise en main OpenCode incluant un flux git

Objectifs d'apprentissage : Se familiariser avec les mécaniques d'OpenCode ET avec un flux git (cloner, lire l'historique, lire une issue) qui sera réutilisé sur le vrai cas.

Voici les différentes étapes à suivre :

1. Cloner et explorer l'historique du dépôt de l'exercice. Par exemple, avec la commande

```
git log --oneline
```

2. Lancer `opencode`, puis la commande `\init`. Vérifier que l'`AGENTS.md` généré décrit correctement le projet ;
3. Avec `opencode`, diagnostiquer et corriger le bug de l'issue 1 comme vous l'avez fait pour l'exercice précédent. Observer le diff, l'accepter, relancer pytest pour confirmer.
4. Écrire la fonction `word_frequency(text)` à partir d'une specification en langage naturel avec `opencode`. Vérifier le résultat à la main sur un exemple avant d'accepter.
5. Écrire les tests correspondants avec `opencode`, lancer `pytest`et juger si les cas limites sont vraiment couverts.
6. Traiter l'issue 2. Il s'agit notamment d'identifier, d'expliquer et de corriger l'issue 2.

3.4 Exercice 3 : se préparer à votre travail à partir d'un vrai dépôt

Avant de passer à votre mission et à votre travail sur un vrai dépôt, nous vous conseillons de discuter en binôme des questions et élements ci-dessous :

- Les issues tenaient ici en quelques lignes. Sur unvrai dépôt, une issue ou un pull request peut contenir des dizaines de commentaires et de liens : comment comptez-vous trier l'information utile avant de formuler votre première demande à une IA ?
- Vous avez localisé le bug dans un seul petit fichier. Sur le vrai dépôt, le code peut être réparti sur des dizaines de fichiers : quelle stratégie allez-vous utiliser pour orienter l'IA vers le bon endroit ?
- `git log -oneline` vous a montré 2 commits. Sur un vrai projet, l'historique peut en compter des milliers : comment allez-vous retrouver le commit exact précédant le pull request qui vous est assigné ?
- La suite de tests de l'exercice jouet ne comptait que 3 tests très ciblés. Sur le vrai dépôt, comment déciderez-vous si les tests existants suffisent à valider une correction proposée par l'IA, ou s'il faut en écrire de nouveaux ?
- Le bug de l'exercice jouet était intégralement documenté à l'avance. Sur le vrai cas, si l'IA propose une correction qui fait passer les tests mais que vous ne comprenez pas complètement pourquoi elle fonctionne, que ferez-vous ?
- vous avez comparé deux modèles sur une fonction triviale. Qu'attendez-vous de différents modèles lorsque vous referez cette comparaison sur une tâche réelle plus complexe ?

4 Travail sur votre mission

Comme expliqué ci-dessus, par binôme, vous allez traiter une tâche réelle de développement logiciel sur un dépôt de votre choix, en utilisant Aristote comme assistant intégré à votre IDE. Une liste de dépôt vous est proposée par mention. C'est une liste de suggestions, mais vous pouvez choisir le dépôt et les tâches par vous-même.

Le principe est cependant de ne pas traiter d'issues encore ouvertes, mais de construire des tâches à partir d'une pull request réelle et déjà fusionnée sur le dépôt choisi. Il faut reprendre le code au commit immédiatement antérieur à ce merge, et travailler à partir de cette version. Vous pourrez alors considérer la pull request fusionnée comme version de référence pour effectuer votre analyse. Un point d'attention concernant les dépôts très connus qui font partie des données utilisées pour pré-entraîner ou fine-tuner les modèles est la mémorisation du correctif plutôt qu'un véritable raisonnement sur le code. C'est une bonne question à se poser lors de l'analyse.

La liste des tâches possibles est :

- **Correction de bug documenté** : reprendre un pull request de correction de bug, avec reproduction du bug puis correction assistée par l'IA.
- *Ajout d'une fonctionnalité avec tests* : reprendre un pull request d'ajout de fonctionnalité clairement spécifiée mais non triviale, en exigeant des tests unitaires générés ou complétés avec l'aide de l'IA, puis vérifiés.
- **Refactoring de code legacy** : reprendre un pull request de refactoring d'un module existant (avec tests de non-régression déjà en place), en observant comment l'IA gère — ou perd — le contexte métier

4.1 Choix du dépôt - Liste indicative

TABLE 1: Dépôts suggérés — Architecture des systèmes informatiques

Dépôt	Niveau	Tâches possibles	Exemple de Pull Request
redis/redis-py	Standard	Client Redis : gestion de connexions/pooling, sérialisation, commandes.	PR #4223 — inversion latitude/longitude dans le helper geo() du query-string builder RediSearch (querystring.py, 1 ligne + test dédié) ; aucun serveur Redis réel nécessaire, le builder est testé hors ligne.
celery/celery	Confirmé	File de tâches distribuée : gestion des workers, retries, routage — bon terrain pour les systèmes distribués. Codebase dense, réserver aux binômes à l'aise.	PR #9869 — app.control.revoke() ne mettait pas à jour immédiatement le statut REVOKED dans le result backend (worker/control.py, ~70 lignes dont tests) ; tests basés sur un backend simulé, aucun broker réel requis.
giampaolo/psutil	Standard	Monitoring système (process, mémoire, réseau) multiplateforme : cas limites par OS, ajout de métriques.	PR #2818 — process_iter() filtrait silencieusement les processus zombies au lieu de les renvoyer, à cause d'une hiérarchie d'exceptions mal gérée (psutil/__init__.py, ~19 lignes, test basé sur mock)
agronholm/apscheduler	Standard	Planificateur de tâches léger, pur Python, zéro dépendance externe pour les tests : bon complément si un dépôt ci-dessus est déjà attribué ou instable.	PR #1130 — CronTrigger acceptait des valeurs day_of_week hors plage et levait un IndexError incompréhensible au lieu d'un ValueError explicite (triggers/cron/fields.py, test paramétré dédié) ; fusionnée très récemment, disponibilité à reconfirmer.
gorakhargosh/watchdog	Standard	Surveillance de système de fichiers, pur Python, testable sur des événements synthétiques sans surveillance réelle : bon repli léger.	PR #1158 — un str.replace() sans limite corrompait les chemins internes lors du déplacement d'un dossier dont le nom apparaît plusieurs fois dans le chemin (events.py, ~40 lignes dont test de régression)

TABLE 2: Dépôts suggérés — IA

Dépôt	Niveau	Tâches possibles	Exemple de Pull Requezst (PR)
fastapi/fastapi	Standard	Servir un modèle via une API, très représentatif du métier MLOps : validation Pydantic, cas limites de routing.	PR #15763 — résolution de route cassée pour un path vide dans un routeur sans préfixe (fastapi/routing.py, ~65 lignes)

(suite page suivante)

Suite du tableau 2 — Dépôts suggérés — IA

Dépôt	Niveau	Tâches possibles	Exemple de PR
huggingface/ datasets	Standard +	Chargement/traitement de jeux de données : bug sur un loader ou une transformation. Attention : certains tests téléchargent des données, à tester hors ligne au préalable.	PR #8243 — mauvaise inférence de schéma JSON sur un fichier commençant par un BOM UTF-8 (json.py, ~24 lignes, aucun téléchargement requis)
optuna/optuna	Standard	Optimisation d'hyperparamètres, en pur Python, installation légère (pas de framework deep learning requis) : stratégies d'échantillonnage, callbacks, cas limites de pruning.	PR #6803. Autres options : PR #6786 ou PR #6771 — biais d'arrondi dans le tirage QMC (_transform.py, ~85 lignes dont tests)
Farama-Foundation/ Gymnasium	Standard	Environnements de RL (successeur d'OpenAI Gym), dépendances légères : bug sur un environnement, wrapper, espace d'observation/action.	PR #1419 — AsyncVectorEnv/ SyncVectorEnv.step() restait bloqué si le nombre d'actions ne correspondait pas au nombre d'environnements (~50 lignes)
scikit-learn/ scikit-learn	Confirmé	Cœur du ML classique : viser les modules purement Python (model_selection, preprocessing) pour éviter les parties Cython. Cas limites de validation croisée ou de transformation de features.	PR #31559 — FeatureUnion échouait avec une erreur cryptique quand un transformer renvoyait un array 1D (pipeline.py, ~30 lignes, aucun Cython touché)
scikit-learn-contrib/MAPIE	Standard +	Quantification d'incertitude / prédiction conforme, compatible scikit-learn, dépôt plus modeste : cas limites de calibration, ajout d'un score de non-conformité.	PR #942 — sélection de seuil non optimale en cas d'égalité, et vérification de monotonicité trop stricte (risk_control/, ~20 lignes) ; regroupe deux corrections, restreindre l'énoncé à une seule si besoin.
gradio-app/ gradio	Standard	Interfaces web pour démos ML, backend très majoritairement Python : bug sur un composant.	PR #13344 — KeyError dans @gr.render quand un gr.Examples imbriqué retire temporairement une fonction (renderable.py, ~75 lignes, 100% Python côté serveur)
scikit-learn-contrib/ imbalanced-learn	Standard	Sur-échantillonnage compatible scikit-learn (SMOTE et variantes) : mêmes familles de tâches que MAPIE/scikit-learn, dépôt plus léger, bon repli si les deux précédents sont déjà attribués.	PR #1177 — message d'erreur peu clair dans SMOTENC avec un encodeur catégoriel qui supprime des colonnes (base.py, ~53 lignes)
online-ml/river	Standard	Machine learning en ligne / streaming, pur Python, installation très légère : bon complément pour varier par rapport aux bibliothèques batch classiques.	PR #2002 — MinMaxScaler ne gérait pas une feature jamais vue à l'entraînement (scale.py, ~35 lignes)

TABLE 3: Dépôts suggérés — Science du logiciel

Dépôt	Niveau	Tâches possibles	Exemple de Pull Request
pallets/click	Standard	Framework CLI (base de Typer) : bugs de parsing d'options/arguments, cas limites de validation.	PR #3805 — les membres de Sentinel ne survivaient pas à copy/deepcopy/pickle, cassant l'identité de singleton (_utils.py, ~60 lignes dont tests)
pallets/flask	Standard	Framework web compact et mature : bug de routing ou de config, petite fonctionnalité, tests de régression.	PR #6096 — parsing naïf de SERVER_NAME/cookies avec partition(" :"), cassé sur une adresse IPv6 (app.py/testing.py, ~23 lignes)
psf/black	Confirmé	Formateur de code (manipulation d'AST) : cas de formatage mal géré, ajout d'une option de configuration.	PR #5148 — indentation de docstring cassée par un expandtabs() à 8 espaces au lieu de la convention 4 espaces de Black (~37 lignes, format de test par fixtures d'entrée/sortie)
pytest-dev/pytest	Confirmé	Le framework de tests lui-même (aspect « méta » intéressant) : bugs sur fixtures ou reporting.	PR #14552 — cache de bytecode non invalidé lors du déplacement d'un fichier de test, fuite de l'ancien chemin dans co_filename (assertion/rewrite.py, ~33 lignes) — mécanique interne de réécriture d'assertions.
python/mypy	Confirmé	Vérificateur de types statique : bugs d'inférence sur des cas précis, discussion typage/fiabilité.	PR #21989 — crash (AssertionError) sur un match avec sujet de type tuple déballé par étoile (checker.py, ~20 lignes).
python-jsonschema/jsonschema	Standard	Validation de schémas JSON : ajout d'un validateur, gestion d'un format non couvert.	PR #1390 — json_path des erreurs de validation mal échappé pour des clés non identifiantes, JSONPath ambigu (exceptions.py, ~9 lignes + 12 tests).
joke2k/faker	Modulaire	Génération de données factices, un fichier par « provider » : pratique pour attribuer un module distinct à chaque binôme.	PR #2396 — récursion infinie dans _safe_random_int quand les deux bornes convergent vers le même entier (providers/python/__init__.py, ~29 lignes) ; corrige 3 issues historiques d'un coup.
arrow-py/arrow	Repli / plus simple	Dates et heures : cas limites de parsing/formatage, fuseaux horaires. Bon calibrage court.	PR #1224 — humanize() mal calibré sur l'échelle du mois, corrigé par une arithmétique calendaire (arrow.py, ~34 lignes) ; la PR retire des @pytest.mark.xfail existants, cas d'école « rouge avant / vert après ».

Table 4: Dépôts suggérés — Cybersécurité

Dépôt	Niveau	Tâches possibles	Exemple de PR
pyjwt/pyjwt	Standard	Bibliothèque JWT (a connu de vraies vulnérabilités historiques, ex. confusion d'algorithme) : validation de signature, gestion des claims, cas limites d'expiration.	PR #1040 — le claim iss n'était pas vérifié en type, une valeur non textuelle passait les contrôles d'égalité/appartenance (api_jwt.py, ~25 lignes + tests)
lepture/authlib	Standard +	OAuth2 / OpenID Connect : validation de tokens, flux d'autorisation, cas limites de scopes.	PR #916 — dans le flux JWT-Bearer (RFC 7523), les claims sub/iss étaient lus avant vérification de signature, sans contrôle de présence/type (client.py/ jwt_bearer.py, ~48 lignes de correctif + tests) ; illustre directement la règle « ne jamais faire confiance aux claims avant vérification de signature ».
pyca/ cryptography	Confirmé	Primitives cryptographiques (API Python ; bindings Rust/ OpenSSL sous le capot) : bugs sur l'API haut niveau, garde-fous de validation d'entrée.	PR #15461 — scan par expression régulière en temps quadratique dans load_ssh_private_key, déni de service par complexité algorithmique sur une entrée sans marqueur de fin (ssh.py, ~44 lignes dont tests) ; bug de classe CWE-1333, dans la couche Python pure, pas le cœur Rust.
certbot/ certbot	Standard	Client TLS/Let's Encrypt (EFF), dépôt mature : gestion de certificats, cas limites de configuration.	PR #10651 — un commentaire dans une directive nginx multi-lignes avec guillemet non apparié faisait dérailler le parseur pyparsing et engloutir le reste du fichier de config (nginxparser.py, ~40 lignes dont test) — niveau confirmé ; testable sans processus nginx réel.
OWASP/ PyGoat	Méthode différente — voir note	Application Django délibérément vulnérable (Top 10 OWASP) : identifier une vulnérabilité connue puis la corriger avec l'IA.	Sans objet — méthode différente du §4.2 (voir note ci-dessous).
PyCQA/ bandit	Standard, périmètre réduit	L'outil d'analyse statique de sécurité Python lui-même : angle « auditer l'outil d'audit », pur Python, installation triviale (pip install bandit).	PR #1337 — le check B608 (injection SQL) exigeait un espace après VALUES, donc VALUES(...) sans espace (SQL valide et courant) n'était pas détecté (~12 lignes, correctif d'une regex)

Pour chaque tâche visée, il faudra préparer votre poste. Par exemple, sur le cas `optuna` (liste IA), pour la PR optuna/optuna 6803, on fera

```
git clone https://github.com/optuna/optuna.git
cd optuna
git checkout ba3fcfb~1   # etat juste avant le commit de correction (ba3fcfb)
```

5 Livrables et analyse

5.1 Journal de prompts

Table 5: Journal de prompts — modèle à dupliquer (Annexe A)

Heure	Demande formulée à l'IA	Réponse / suggestion obtenue (résumé)	Décision et justification (accepté / modifié / rejeté)

5.2 Fiche d'analyse critique, prise de recul

- Temps estimé sans IA vs temps réel avec IA
- Nombre d'interactions avec l'IA jugées utiles / peu utiles / contre-productives
- Erreurs ou hallucinations identifiées (au moins 2, décrites brièvement)
- Un choix de code que vous n'auriez PAS fait sans l'IA, en bien ou en mal
- Ce que vous referiez différemment dans votre façon de solliciter l'IA

Vous discuterez aussi les aspects ci-dessous dans votre note de prise de recul.

- **Productivité**
  - Sur quelles parties de la tâche l'IA vous a-t-elle réellement fait gagner du temps ? Sur lesquelles vous a-t-elle ralenti·es (relecture, corrections, prompts répétés) ?
  - Votre estimation « sans IA » du temps nécessaire était-elle proche du temps réellement passé avec l'IA ?
- **Fiabilité et qualité**
  - Quelles erreurs ou « hallucinations » avez-vous rencontrées ? Comment les avez-vous détectées ?
  - Auriez-vous détecté ces erreurs si vous n'aviez pas eu de suite de tests ou de revue de code ?
- **Autonomie et compétences**
  - Y-a-t-il des moments où vous auriez pu écrire le code plus vite vous-mêmes qu'en formulant une demande à l'IA ?
  - Quel risque voyez-vous, à long terme, à déléguer systématiquement certaines tâches à l'IA ?
- **Éthique, souveraineté, responsabilité**
  - En quoi le fait qu'Aristote soit une infrastructure souveraine et mutualisée entre établissements change-t-il (ou non) votre rapport à l'outil, comparé à un service commercial grand public ?
  - Qui est responsable si un bug généré par l'IA et non détecté part en production : le développeur, l'éditeur du modèle, l'entreprise ?
  - Que faudrait-il mettre en place dans une équipe professionnelle pour encadrer sainement l'usage d'un tel outil ?